Отчет по тестированию дипломного проекта BuildMap

1. Краткое описание проекта

BuildMap — это система indoor-навигации по зданию с веб-админкой для настройки карт, этажей, точек и связей между ними.

Пользовательская часть строит маршрут между точками внутри зоны и отображает пошаговые подсказки.

Backend реализован на Spring Boot, frontend состоит из двух React-приложений: `admin_panel` и `buildmap_nav`.

2. Артефакты этапов 1 и 2

Mind map:

- `tests/mindmap/buildmap-testing-mindmap.drawio`
- `tests/mindmap/buildmap-testing-mindmap.png`

Тест-кейсы:

- `tests/test-cases/test-cases.csv`

Набор содержит 20 кейсов в формате:

- `ID`
- `Название`
- `Шаги`
- `Ожидаемый результат`
- `Приоритет`
- `Тип`

3. Обоснование инструментов и типов тестов

Использованные инструменты:

1. `JUnit 5` — основной framework для unit и integration тестов в Java.

2. `Spring Boot Test` + `MockMvc` — проверка REST API, валидации и security-ограничений.
3. `Mockito` — изолированная проверка алгоритма маршрутизации на unit-уровне.
4. `Maven` — единая точка запуска тестов.

Выбранные уровни тестирования:

1. `Unit` — проверка ключевой логики алгоритма маршрута (`Dijkstra`).
2. `Integration` — проверка API, бизнес-правил и авторизации.

Почему выбраны именно эти уровни:

1. Лучше всего подходят под текущий стек проекта (Spring Boot + REST + JPA).
2. Позволяют быстро и воспроизводимо проверить критичные сценарии из `test-cases.csv`.

4. Примеры ключевых тестов

Ниже приведены 3 важных автоматизированных теста.

Пример 1. `TC-002` Telegram login с валидными данными

Файл: `api/src/test/java/com/buildmap/api/integration/ApiSecurityIntegrationTest.java`

```
@Test
void tc002_telegramLoginWithValidData() throws Exception {
    long authDate = System.currentTimeMillis() / 1000;
    long telegramId = 222000111L;
    String firstName = "Test";
    String username = "tc002_user";
    String hash = buildTelegramHash(authDate, telegramId, firstName,
username);

    String body = """
            {
              "id": %d,
              "first_name": "%s",
              "username": "%s",
              "auth_date": %d,
              "hash": "%s"
            }
```

```
            """.formatted(telegramId, firstName, username, authDate, hash);

    mockMvc.perform(post("/api/auth/telegram")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(body))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.token").isNotEmpty())

.andExpect(jsonPath("$.user.telegramId").value(String.valueOf(telegramId)));
}
```

Смысл проверки:

1. Валидный Telegram payload должен приниматься backend.
2. Система должна вернуть `token` и данные пользователя.

Пример 2. `TC-014` Запрет связи между точками разных зон

Файл: `api/src/test/java/com/buildmap/api/integration/ApiDomainIntegrationTest.java`

```
@Test
void tc014_connectionAcrossAreasIsRejected() throws Exception {
    User user = saveUser("Case User", "tg_tc014", Role.USER, false, false);
    String token = jwtService.generateToken(user.getId());

    Long areaA = createArea(token, "TC014 Area A");
    Long areaB = createArea(token, "TC014 Area B");
    Long floorA = createFloor(token, areaA, "A1", 1);
    Long floorB = createFloor(token, areaB, "B1", 1);
    Long fromId = createFulcrum(token, floorA, "A-Point", 0.2, 0.2);
    Long toId = createFulcrum(token, floorB, "B-Point", 0.7, 0.7);

    mockMvc.perform(post("/api/fulcrum/" + fromId + "/connection")
                    .header("Authorization", "Bearer " + token)
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(Map.of(
                            "connectedFulcrumId", toId,
                            "distanceMeters", 6.0,
                            "difficultyFactor", 1.0
                    ))))
            .andExpect(status().isBadRequest());
}
```

Смысл проверки:

1. Зафиксировано важное бизнес-правило целостности графа.
2. Нельзя соединять точки из разных `mapping area`.

Пример 3. `TC-020` Маршрут на большом графе

Файл: `api/src/test/java/com/buildmap/api/integration/ApiDomainIntegrationTest.java`

```
@Test
void tc020_routeOnLargeGraphReturnsWithinReasonableTime() throws Exception {
    User user = saveUser("Case User", "tg_tc020", Role.USER, false, false);
    String token = jwtService.generateToken(user.getId());
    Long areaId = createArea(token, "TC020 Area");
    Long floorId = createFloor(token, areaId, "L1", 1);

    List<Long> nodeIds = new ArrayList<>();
    for (int i = 0; i < 20; i += 1) {
        double x = (double) i / 19.0;
        nodeIds.add(createFulcrum(token, floorId, "N" + i, x, 0.5));
    }
    for (int i = 0; i < nodeIds.size() - 1; i += 1) {
        addConnection(token, nodeIds.get(i), nodeIds.get(i + 1), 1.0, 1.0);
    }

    Long startId = nodeIds.get(0);
    Long endId = nodeIds.get(nodeIds.size() - 1);

    assertTimeoutPreemptively(Duration.ofSeconds(5), () ->
            mockMvc.perform(post("/api/navigation/path")
                            .contentType(MediaType.APPLICATION_JSON)
                            .content(objectMapper.writeValueAsString(Map.of(
                                    "startFulcrumId", startId,
                                    "endFulcrumId", endId
                            ))))
                    .andExpect(status().isOk())
    );
}
```

Смысл проверки:

1. Проверка устойчивости маршрутизации на увеличенном графе.
2. Контроль времени ответа в интеграционном сценарии.

5. Инструкция по запуску тестов

Основная команда:

```
cd api
mvn test
```

Примечание по окружению:

1. В `pom.xml` задан `java.version=21`.
2. Для локального окружения с Java 17 использовался запуск:

```
mvn "-Dmaven.compiler.release=17" test
```

Результат последнего прогона:

1. Выполнено 21 тест.
2. `Failures: 0`, `Errors: 0`.
3. Статус: `BUILD SUCCESS`.

6. Вывод: результат тестирования, найденные проблемы и сложности

Что дало тестирование:

1. Подтверждена корректность критичных бизнес-правил по зонам, этажам, точкам и связям.
2. Проверена безопасность ключевых API-операций (роль, токен, blocked user).
3. Подтверждена работоспособность построения маршрута в обычных и граничных сценариях.

Найденные проблемы/наблюдения:

1. В логах присутствуют служебные debug/console сообщения (например, из Telegram validation), что нежелательно для production-окружения.
2. Тесты до этого проводились вручную, полученный результат, ожидаемый в силу предыдущих ручных проверок.

Сложности:

1. Для воспроизводимого прогона понадобилась аккуратная настройка тестовой SQLite-базы и изоляция данных между тестами.